

The prompt is now the fundamental unit of engineering. And with agents, every prompt you create becomes a force multiplier. One well crafted prompt can generate tens and hundreds of hours of productive work. One bad prompt compounds failure at the same rate. So how do you consistently create prompts that deliver 2, 5, 10, 100x returns on your time investment? The answer starts with a senior engineering mindset shift. You have to ask yourself, who am I creating these prompts for? It's not just for you. It's not just for your team. We're now engineering for three audiences. You, your team, and your agents. This is the stakeholder trifecta for the age of agents. This is the agentic shift that most engineers haven't yet made. The winners here are building libraries of reusable, battle tested agentic prompts with composable sections that work like Lego blocks. You can swap these in and swap them out when and if you need them to scale your engineering impact with compute. But things have changed. We're now agentic prompt engineering, and every prompt you write has the ability to call tens and hundreds of tool calls and run from minutes to hours. We're going to understand the seven most powerful agentic prompt formats and most importantly, the exact, exact sections that make them up. We're talking about the most useful sections for your prompts and how challenging they are to operate you, your team, and your agents. Let's level up our agentic prompt engineering abilities by mastering the prompts and their composable sections. The next prompt we're going to look at is the workflow prompt. This prompt represents a sequential workflow with a bunch of new additional sections that you can work with. Out of all the agentic prompt formats we'll look at, this is easily the most important one. Why is that? It's because of this workflow section. Let's go ahead and grade the workflow section and the workflow prompt just to set the stage to showcase how powerful and how useful this is. So workflow and throw it into the S tier usefulness. This is the most useful section and we're going to throw it in C tier difficulty. It's actually not that challenging to use the workflow as you'll see in a second. We're also going to take the workflow prompt and place it in A tier usefulness and C tier skill requirement. So these are both very useful, very powerful, and so why is this so powerful? Let's look at one of the most basic versions of this in the prime command. So this is something you've likely seen me use. The prime command sets up your agent to gain an understanding of the code base so it can start working. But here we have several sections to break down. So we have this brand new metadata section, and this is specifically metadata for Claude code. But other agents and tools you work with will have their own version of this. But here we just have a simple description. We have our same title, we have our purpose here. And below we have two new sections, right? We have the workflow and we have the report. The workflow section is exactly as you think. The key here is that it's a sequential list of tasks that you want

your agent to execute to get the job done. So whenever we run our prime command, we want our agent to do just this, right? We can, of course, spin up an agent to run this whenever we want to. It's going to do just, just these two things, right? And we're setting it up in the purpose execute the workflow and report sections to understand the code base and summarize your understanding. So let's go ahead and grade the metadata in the report section. So metadata is important, but it doesn't actually do a whole lot for your prompt, for your team, and for your agent. It's not completely useless. But out of all the sections and out of all the prompts we'll look at metadata belongs in C tier usefulness and C tier skill requirement. Metadata lets you do a couple powerful things you'll see in upcoming prompts, like specify the exact tools or the exact models that you want. But other than that, it's not all that valuable. And we also have the report section. Now the report section is going to be quite a bit more valuable. The report section lets you change the way your agent responds to you. Anything from running strictly in JSON output, YAML output, or to report in a specific structure, or to include specific information in the report. It all goes under the report section, right? This is your output section. This is your format section, okay? And to be super clear, you know, you can call these sections whatever you want. I'm using the most consistent, the most information dense keywords to represent every section, okay? Out of the tens of thousands of prompts I've seen and used and run, these sections keep coming back over and over and over. So no matter what you call them, they're still going to be there. As you scale up the level and the capability of your agentic prompts, you can see here this is the foundation for the workflow prompt. All right, we're now executing a series of steps that can get very complex. That can call many, many tools. All right, so let's look at another prompt. Right? We have this build prompt. Let's check this out. And you can see here a little bit more metadata. Right? Our build prompt is only able to use read, write and bash tools. And it has an argument hint. The argument, of course, leads to a really important section, a super underappreciated section, the variables. Okay? And so, you know, before we get to variables, everything else is the same. We have the title, we have the purpose. In our purpose here, we're being a bit more prescriptive about what to do. Exactly. And then we have our workflow and our report section. Variables is very, very important. This is where your prompts become massively more valuable. A lot of the work we do is repeat work that runs in steps, sequences of steps, Right? This is what the agentic workflows right now are solving for us. And you can throw agentic workflows right into one of these workflow prompts. Right. You can also scale them up with ADWs, as you've seen and as we've discussed at length in tech. But the variable section is ultra important because now we're parsing the claw code arguments syntax here and in our prompt. When this gets executed, this is going to update so that whatever the value is here, Right? Right. So path to plan. And then check this out. We are referencing the name of our variable throughout our prompt. Okay? Variables let you pass information into your prompts. Here we're building up our own easy to understand natural language variable syntax that we can reference throughout the

prompt. Okay. And powerful modern language models running in these agents. They're powerful enough to know exactly what you mean when you keep referencing it, especially in this syntax, consistent syntax. Not only is this great for your agent, it is of course, great for you and your team. Look at how easy this is to understand. Right? We have a variable coming in and this is the path to the plan. Okay? So very powerful stuff here. This is a dynamic variable in contrast to a static variable. Let's go ahead and grade this. Right? How powerful are variables? How useful are they? These, of course, are very useful. This is going to be a tier usefulness and we're going to put this in B tier difficulty, Right? B tier skill requirement. Because now we're referencing variables both static and dynamic across your prompt. Okay? And you can see here, this is just a simple 23 line prompt. This is going to scale up in difficulty, right? Let's go ahead and look at a bigger prompt. Let's look at quick plan. So when I'm working with prompts, I like to just always open it up in full and then collapse to the second level so that you can see all of the sections consistently. You can quickly see the composable sections in this prompt, okay? This immediately gives you a good idea of how complex and how powerful the prompt is. So you can see here we have more metadata, we have the model now, argument, hint, description and allowed tools title. And then we have our purpose. And so notice the language here, right? Create a detailed implementation plan based on the user's requirements. We're just like very directly talking to our agent, all right? And so by contrast, by having this very kind of dry, direct language, you and any engineer on your team can come to this prompt, collapse everything, and just read this upper section to understand exactly what this does, right? And what the capabilities are. Then you can just dial in one piece at a time, right? So what are the variables here? You can see here we have that user prompt coming in as the argument, and we have the plan output directory. Now, this is different, right? You can see how different this is. This is a static variable. This is a variable we're placing right in the prompt that doesn't change. You can't pass anything in to overwrite this. This is a powerful static variable versus a the user prompt. This is a dynamic variable, right? That changes as you pass in work. This prompt is going to write to the specs directory, right? And if you want to, you can change this to be anything you want. So say you like PRDs, right? This will write to the PRD directory, right? All you have to do is update this, save it, and that's it, right? You don't have to keep referencing this everywhere. And of course, we can copy this and do a search on this variable, right? And throughout you can see this getting referenced. Notice how much time I'm saving by having just a single static variable here that we can reference throughout the prompt. And also notice how easy it is to understand this. Great. Prompting is great. Communicating. Okay, so what we're doing here is quickly communicating to ourselves, to our future selves, to our team, and of course to our agent. Okay, so user prompt is going to come in. It's going to output to the specs directory, right? And very quickly, just two sections, you know exactly what this is going to do. We do have a new instruction section here. Let's think through this, right? How is this different than the workflow section in your

agentic prompts? So the workflow is the step by step play of exactly what you want your agent to do. Oftentimes your agent, if it's a cloud code level agent, it's going to come through and create a list, a concrete list inside of its to dos, inside of its plan of your workflow. So this maps very, very well to modern agent encoding tools as specified below. You can see here we're creating a specific output type. We're almost bleeding into the examples section, but this is just a report. Okay. So we're just specifying the output format we want. So the workflow is a play by play step of what we want done. And then the instructions are surrounding information on the individual steps in the workflow. So this is a great place to just come in and add auxiliary information on how your workflow should work. Now, to be clear, oftentimes you can just combine these. Right. And you can create, you know, nested bullet points underneath your workflow steps in upcoming prompts. You're going to see that. Exactly. But you know, you can use instructions to just quickly tag on useful information to aid your workflow. Okay, so we have one new additional section here, right? We have the instruction section. So how does this rank? This is going to be right below the report. Right. So probably as useful and as challenging to use as the report section. Right. This is a bullet list of additional information that aids the workflow. Okay, so it's not ultra difficult to use, but oftentimes you don't need the instructions, you can just use a workflow. All right, But I wanted to call this out. These are two distinct patterns. And especially when we start talking about system prompts versus user prompts, you're going to see that the instructions are going to be a lot more valuable for your system prompt versus your workflow. When you're writing these reusable agentic prompts, this of course is going to be a lot more useful for the actual play by play that you want your agent to run in a repeat fashion. We have the prime tier list. Okay. And I just want to show this off. We have one additional section here. And we have no metadata, so notice this format. Right. Title, purpose. And then we have code based structure. This is a distinct section. We have our workflow. And you can see our workflow is getting a little bit more complex here. We'll break that down in a second. And we have a report. Okay, so just a kind of super simple blank report. Let's go ahead and talk about the code base structure. What is this new section? Another way to think about the Code base structure section that you can swap in and out of your agentic prompts. This is really a context map, right? This is the real name of the code base structure. But normally this is just as we're engineers working in code bases, this is usually just called code based structure. And the distinction here is that you're not saying you're not telling your agent to read these files. You're giving them a quick map of where the files are, what the structure looks like of the relevant code. Right? We have a prime tier list here. So we're priming against a specific part of our application and we're specifying exactly what every file does. You know, at 100 Foot View, there's our previous prime command that ran. We can of course clear that out and then prime the tier list. And now the tier list is going to run not these files, but in our workflow, we've specified exactly what files we want to read. Okay, Read these files exclusively. We have to be a little bit stronger with our

language. Here we have an information dense keyword important that has more value to our agents and to the models that run them. And so you can see here we're only reading these specific files, but our agent now has in its context window, thanks to our prompt engineering, it has a map of how to quickly find relevant files. Okay, so instead of having to search this manually, run additional tools, chew up additional tokens, it has a quick, simple map. Okay, and again, code based structure. You and your team can quickly look at this and understand it. Let's go ahead and rank this one. This is not as useful because of course, agents can do this. The big benefit here is that it's faster, right? This speeds your agent up. It helps your agent understand more quickly. So we're going to throw this right under metadata, right? Code base structure is about C tier usefulness and it's not that challenging to use. We're just giving a code based structure and we're asking for a little bit of detail. All right, but now the important interesting thing here is our workflow. If we look at our workflow via regex, we have a more built out list. And so your workflow can be as detailed as you need it to be. So as we cycle through this, you know, you can see here we still have our numbered list, but underneath this we're adding additional details, right? We even have a list with a list underneath. Okay. And so this is very common. You know, you've seen, you've likely seen prompts like this where the workflow Step has decent amount of information underneath the steps. The point is to keep these clear and concise. We could probably clean up this section a little bit and maybe turn it into an instruction set, but this is fine, right? And then at the end here, we run what our original start prompt does at the end of this prompt, right? In step six, we boot up the application, install dependencies, kick it off, open the browser. Okay, so you can see here we're scaling up the value, we're scaling up the use we can get out of our prompts by using these consistent and swappable prompt structures. The workflow prompt is very, very powerful. All the prompts stack up on each other and the workflow prompt is an absolute pillar for all subsequent prompt levels. It's important to note here, by the way, that every one of these prompts, if you open up build, every one of these prompts has a kind of consistent structure, right? It has the inputs, it has the workflow, and it has the output. Input, workflow, output. This is a very consistent three step pattern you can use to think about how to design and build your prompts, right? Inputs. We have the actual work that your agent does and then we have the report. Now, obviously all of this information is important for your agent, but the input and the output are specifically useful for you and your team. Now, we can quickly know what's coming in and what's going out, right? Of course, we'll be spending a lot of time on the workflow dialing this in making it do exactly what you want. But this is a useful framework for thinking about how to build great reusable agentic prompts consistently input workflow output. Let's move on to level three of our agentic prompt format. At the next level, we continue building off our existing sections. Let's open up Build md. You know this prompt is going to come up over and over. It's a very powerful, very simple prompt. But you can see there's something going on here. On line 17 we have flow control. Okay, so level three is the control

flow prompt. So we're building up on our workflow section and we're adding this new capability. Remember, every prompt format is about the capability that it offers you and your agent. The control flow prompt lets us run conditions, loops and early returns. You can see here if no path to plan is provided, immediately stop and ask the user to provide it. So we can of course, very quickly test this. You can see here our prime tier list finished. It's got a great, great detailed layout for us on this specific feature. Check out our extended lesson on context engineering. To understand context priming can be more valuable than using a memory file. But anyway, let's go ahead and clear this out and let's just kick off build and we pass in no path to our plan, right? Our variable isn't there, so right away our agent sees this and it stops, right? This is the equivalent of an early return asking for feedback, right? We need some information. We cannot proceed unless we have a path to plan. This is, of course, the most simple version of flow control inside of your prompts. You can imagine this getting a lot more complex. The most popular extension of this will be, of course, condition conditionals and loops. Let's open up the Create image prompt here, where we're creating an image. We're going to run that exact same understanding workflow, right? We collapse everything on level two and we just analyze the prompt top to bottom, right? We have the metadata section, we have the title. We have a kind of a bad tone of voice purpose. But it's fine. This will get through. We're saying this command generates images. Really? You want that direct tone of voice. You'd say something like this. Generate images based on the provided prompt using replicate MCP server. We're talking to our agent. We have a request targeted Tor agent. All right, you can see here two variables. Let's open up our variables. And you can see here we're using a new syntax, and we also have a bunch of static variables with a couple bullet points. Okay, so let's analyze this, right? So we have word plus colon. And this gives us our variable format. So what's going on here? Right? We have several variables, and we're using claude code's positional argument for our first arg and then our second argument, right? So of course, this would look like this slash create image. And you can see here we would pass in something like some file with image prompts, and then we pass in a numeric number, positional arguments. You know exactly how that works. We have three additional static variables. Okay, so very powerful stuff here. We're starting to use more static variables that we can reference throughout our prompt, right? This is very useful for agents and to communicate value to yourself, right, in the future and to your team, right? So image output directory. We can quickly just see where this is referenced. And we can see, you know, the model we're using. We can quickly just update the model that we're using to generate the image. And of course, we can update the aspect ratio, so on and so forth, right? All this stuff, we can add arbitrary static and dynamic variables here, right? But the interesting thing here is in the workflow, in these control flow prompts, you can, of course, with natural language request loops. Okay, so we're saying important, then generate number of images, right? We're referencing a variable using the image generation prompt following the image loop

below. So we have this XML block to add additional structure. We're communicating to our agent when the image loop starts and stops. We're also, of course, communicating to our team where the loop is. We're operating in natural language. We want consistent patterns throughout our prompts for the trifecta, right? And so just more workflow details, right? Top to bottom, use this tool, pass an argument, use aspect ratio, wait for completion, and then throughout, we're referencing our variable. So you can see here this is getting more and more complex, right? We're doing more and more with our prompts, of course, have the report generates images at scale for us in a loop. Let's go ahead and rank this. When it comes down to it, you know, the conditional flow prompt, I would say is about as powerful, probably a little bit more powerful, more useful than the workflow prompt, right? There's probably a little bit above this. It is going to be more challenging, though. It requires more skill to think through and analyze the workflow. Step in your prompt to understand the control flow that your prompt is working through. All right? So for that reason, we're going to place this in A tier usefulness and B tier skill requirement. Let's crack open the edit image and you can see a very similar format. You know, nothing to change here, right? We have a consistent process, right? We have metadata at the top, title, purpose, variables, workflow, and notice here, no report section. Remember, every one of these sections is swappable. Use them only if you need it. If you don't care about the output format of your prompt, right? Remember the three step workflow, input, workflow, output. If you don't care about your output, or if you don't need variables coming in, don't add the section, okay? This is the key to writing great prompts. Only add what you need, right? These are composable pieces of your agility. Okay, so the edit image has no output format. And so you can see a same deal here. If we open everything up, we have variables and we have a workflow, and our workflow has a loop embedded inside of it. All right? And we also have a stop command here, right? A couple conditionals, you know, if the token's not available, exit Immediately if. If the base64 command isn't available, exit immediately. Right? This prompt does a specific thing in a specific workflow very well. With a loop embedded. All right, that the control flow prompt. Now we continue to scale up what we can do with our agents while properly communicating to the trifecta. Let's go to level four. So at level four, we have the delegation prompt, right? The delegate prompt. This is a prompt that kicks off other agents to do work. Notice how distinct every one of these prompts are. They give you a new advantage, right? A new capability. Key section here. If you're going to be delegating, you're going to want variables that you can pass into your sub agents, okay? So that's the only section tweak that you might make. Let's go ahead and open up parallel subagents once again. You know, same process collapse. Check out the format. There's our metadata title, purpose. You know, this workflow should feel repetitive. And that's what you want, right? You don't want it to be novel every time you open up a prompt that consumes your engineering cycles, that consumes your ability to work through more engineering work and to generate value, right? You want to be thinking less about

the stuff that can be solved with consistency. Right? Consistency is the greatest weapon against confusion for both you and your agent. Okay. This is a key idea in both our extended elite context engineering lesson and in this lesson, right? We're clearly communicating. We want to reduce confusion to the very minimum. You want to be thinking as you're writing these prompts. You know, you want to be thinking, if I handed this to a coworker, could they complete this work top to bottom? Okay? If that's true, you've probably written a high quality prompt and imagine you hand them 10 additional prompts prompts, right? Don't change the prompt format every single time. This is why this consistency is so important. All right, so if we open up variables here in our parallel subagents, you can see we have two positional dynamic variables, prompt request and count. Right? And you can quickly understand this prompt by reading the purpose right file workflow, launch count in parallel to accomplish tasks detailed in prompt requests. All right? And then in the workflow, step by step, play nested bullet points, nothing fancy, very consistent. You can see, just referencing variables throughout. But so what is the key advantage of this, this type of agentic prompt format? We are handing off work to compute. We're having our agent fire off agents. So there are many ways to use this. If you've Already taken the elite context management extended lesson. You already know that there are only two ways to context Engineer R and D. And here we're using the D of that framework, right? So we can kick this off, right? If we just clear this agent out parallel. And I'm just going to paste in a prompt that I have here, right? I'm going to paste this in extract information we'd want to add to our claw MD that we want all future agents to know about. Essentials only present bullet list. Do not modify the file directly. And we want three. Let's say let's kick it up to five. This is the advantage of spinning up more compute, right? You just bump up the number of instances, the number of agents you want to fire off to accomplish a task. And so what's the advantage of running a prompt like this? Right? LLMs agents, they're non deterministic. So when you ask multiple agents to do something, they're going to return with different results. Okay. And you can see here we're firing up five agents focused on a different aspect of the code base. So our primary agent is instructing the sub agents, right? Our primary agent is the prompt engineer for our sub agents. And this is where things get really tricky, right? This, you know, agent delegation is very powerful, but it's hard to use. Let's go ahead and grade this right now. So we have the delegate prompt and I'm going to go ahead and throw this into S tier usefulness while being in a tier skill difficulty. Okay. This is an advanced technique. Why is that? It's because we're managing, managing multiple instances of agents and we're writing prompts that write prompts, right? We are specifically saying, right, we're talking to our primary agent and step two of our workflow, we're saying design agent prompts. Create detailed self contained prompts for each agent. Include specific instructions, define clear output expectations. Okay. We're really kind of building out that input workflow output three step structure for our primary agent to pass into all of our sub agents. This prompt is very powerful. We're spinning up more compute. That in itself



can build out, you know, entire prompts and agents. So this is a very powerful concept. This is the future of engineering. Once you build up great agents that perform for you and get work done, the next question is how many agents can you spin up? How much work can you do? How much compute can you use? Right? And of course, that's what TAC is all about. That was a big, big theme inside of tack. Spinning up these Pipelines of agent that operate with and without us via Outlook systems. Big idea. All this prompt engineering, all this agentic prompt engineering is just about tapping into that better than anyone. So for that reason, this is S tier, ultra useful, ultra valuable. Very hard to use though, right? You need to be conscious about the flow of information between your agents. So that's just one way to use this. Right? You can see we're getting that response flowing back in here. We can kick up another agent here and let's go ahead and run our load AI document. Let's go ahead and open that up. Load AI docs. So once again with great agentic prompt engineering, we're just going to collapse everything. Understand things one step at a time, right? Metadata, title, purpose, input, workflow, output. Okay? Every single time. Consistency beats complexity. Consistency makes you faster, okay? Makes your team faster, and it makes your agents faster. When you find the working formula, and guess what? I'm handing you the working formula right now with these prompt formats. All right, so you can see here, variables. We just have one static variable for this load AI docs delegation prompt. We are going to delete old docs after 24 hours. And our AI docs here, we have documentation for our agents. And if we open that up, our agent has just deleted them. All right, it's just deleted all these and now it's going to reload them because they were older than 24 hours. Right? You can see here, same deal, we're going to loop where we have a explicit, you know, a new on the fly, information dense keyword that we can reference throughout our project prompts. We have a loop and we're actually having our agent call a sub agent directly. Okay. And so you can see this, right? We have five doc scraper agents scraping the documentation and writing it to a new file. We should see some of these come in here by now. Yep, there's our documentation coming in inside of our AI docs. All right, so our agents are bringing those in one by one. We are delegating, we're offloading course once again, this is the D and the R and D framework of great elite context engineering. So this is ultra powerful. And then we have a, a simple but specific report format. And you're going to see our primary agent return in this exact report format. Okay, we have one more prompt here, the background prompt. And you know, notice how every previous level stacks up, right? You can pull in whatever levels you need into the highest level again, that you need. Right. To solve the problem here, we're delegating Agents, we're using control flow, right? We have conditional statements, you can see here several ifs in our workflow. And of course, this is running in the most powerful section of all your prompts, the workflow. So this is great. You can see our agent returning in that exact format. Right? AI docs report right here. Successor failure, URL markdown path. There it is. So we have massive control over this agent. Right? We knew exactly what was going to happen when we

kicked off this prompt. Okay. And this is the key for transitioning to Outlook agentic code. And you need to know what your prompts are going to do. And the best way to know is to be consistent and of course to test and to prove out your agentic prompts. Or. Right. One more. Here we have the background prompt. Once again, you know, I hope you're getting annoyed and, you know, kind of bored of of this workflow, but this is how you write great prompts at scale over and over and over for you, your team and your agents. You stay consistent. Okay? Metadata, title, purpose. Right? We're using that direct language. Run a cloud code instance in the background to perform tasks autonomously while you continue working. Okay. Purpose clearly defined. One or two sentences, variables. Here you can see we have three dynamic variables. And you. This is going. We covered this in our elite context engineering. I'm not going to run this, but if we open up the instructions, we have some, you know, auxiliary important information for the workflow. Okay. And the workflow is of course, the step by step play. This one gets really complex because we start detailing exactly how to kick off a brand new agent via the cli and we're actually updating the system prompt in a very massive way here. We'll talk about system prompts versus user prompts in a moment. But you can see here this agent is going to kick off another primary agent in the background, and this agent is going to continue to report to a report file as it works in the background using whatever model we specify. All right, so this is the delegation prompt, right? The delegate prompt. You can have your agent pass off work to other agents. Okay, let's move up the stack again. Let's add more power to every one of our prompts. Let's move up to level five. This is our agentic prompt tier list. Now, all the big value is in this top left corner here, right? It's in the workflow. And the workflow prompt, I highly recommend you push to at least the B tier level. If you get to B tier in your skill of a genting prompt engineering, you're going to capture most of the value. A key item that I really stress inside of tack, you're going to want to push to understand how to write template meta prompts. Because once you unlock this S tier, skill and usefulness level, these template meta prompts at level six here, your agentic engineering velocity scales up very, very quickly, right? For you, your team and your agen, specifically because you've templated your agentic engineering. But really, you know, 90% of the time for most engineers, all we need is one of the seven levels of agentic prompt formats, and then we need to learn how to interchange the right section, right? The kind of swappable section, the swappable Lego blocks throughout the prompt that you're writing with the most important section by far. You know, I'm, I'm highlighting this over and over for a reason. Most important section you'll write is the workflow, right? Your step by step plan play for your agent, for your agentic prompt engineering. There are two big ideas here that we've, you know, hit over and over and over. Communicate extraordinarily well to your agents for the best results. Secondly, use consistent prompt formats and they're interchangeable sections so that you can crud reusable prompts for you, your team and your agents. Okay? You want to be creating, reading, updating and deleting your prompt prompts at light speed at the right level, usually

level three. Level four is all you need. Get that great, powerful workflow step built inside of your prompts. Be consistent with your reusable prompts so that you can fire your prompts off over and over and over and reduce confusion for yourself in the future, for your team, and most importantly, for your agents. Your agents will be able to reason about many different prompt structures. Okay, but engineering, engineering is not just about your agents. It's not just about you. It's not just about your team. You need to hit the trifecta, all right, the stakeholder trifecta for the age of agents. There's a new user, there's a new profile, there's a new type of consumer of the work that you're doing. It's your agent. So we need to be thinking about this trifecta. Consistency is king when it comes to writing great agentic prompts. There are other sections that we didn't work through. There are, there are other ways to express these sections. That's not what's important here. Okay? The key idea here is that you have swappable Lego blocks that are the Sections of your agentic prompts that each have distinct uses and capabilities that you can use for repeat success in your agentic engineering. Okay, the prompt is the fundamental unit of engineering. Invest in your prompts for the trifecta to achieve asymmetric engineering in the AG age of agents. Okay, context engineering is very important, but your prompts kick off and dictate everything you do. So here's your next move. Here's what I recommend you focus on next. If you haven't finished tack, finish tack. That is more important than context engineering is more important than agenda prompt engineering that sets up the future of engineering. It helps you move toward becoming an irreplaceable engineer in phase two of the generative AI age. Okay, so take tac. Next, I'm going to recommend you go all the way to the third level and jump into writing customizable agents for your domain specific use case. For your domain specific problems, I recommend you check out the Claude code SDK Mastery to build your own specialized agents. TACT is all about scaling up what you can do with agents. It's about getting out the loop. It's about exiting this simple back and forth prompting experience with these agents so that you can scale up what you can do by building pipelines that do one thing extraordinarily well. Use these seven levels of agentic prompt formats to guide your agentic engineering. Use the sections underneath interchangeably and focus on getting use out of your agents. Right? Start at the lowest level. Just throw it into a level 1 ad hoc high level prompt and then when you need to scale it into a workflow prompt, this is going to be where you get the majority of your prompt output for from the step by step play of what your agent to do. After that you can scale up and stack up your capabilities, leading you all the way up to the powerful template meta prompt. The prompt that builds the prompt. The agent that builds the agent. This is how you scale hard and fast into the age of agents. Great work here. I'll see you in the next extended lesson.